Writing skills *for Claude Code* agents.

Skill features and templates for AI agents

A practical authoring guide, extracted from a working QA agent setup

Canache Gabriel
QA Engineer
Centric Romania

Version 1.0 · October 2026

centric

Contents

1. Introduction 4

1.1 Agent and skill . . . 4

1.2 How to read this guide . . . 4

2. Folder structure 5

3. Skill features 7

F1 Description contract . . . 7

F2 Role in the flow . . . 7

F3 Context block . . . 8

F4 References table . . . 8

F5 Output templates . . . 8

F6 Scripts and helpers . . . 9

F7 Command inputs . . . 9

F8 Tool scoping . . . 9

F9 Sources of truth . . . 10

F10 Numbered workflow . . . 10

F11 Progress tracking . . . 10

F12 Verification . . . 11

F13 Bounded retries . . . 11

F14 Final report . . . 11

F15 Human approval gate . . . 12

F16 Stable ids and hand-off . . . 12

F17 Idempotent write-back . . . 12

F18 Guardrails . . . 13

F19 Lessons . . . 13

F20 Worked example . . . 13

4. Templates 14

4.1 SKILL.md . . . 14

4.2 Agent file . . . 15

4.3 Reference file . . . 15

4.4 Output file with an approval gate . . . 16

5. Agent configuration and access 17

5.1 Where agents are defined . . . 17

5.2 Configuration fields . . . 17
5.3 How the QA agents are configured . . . 18
5.4 How access works . . . 18
5.5 Giving an agent access to an MCP server . . . 18
5.6 Giving access to files and folders . . . 19
5.7 Giving access to commands . . . 20
5.8 Permission modes . . . 20
5.9 Where settings live . . . 21
5.10 Example: locking down qa-bug-reporter . . . 21

6. Commands **22**

6.1 Adding an MCP server . . . 22
6.2 Calling an agent . . . 23

7. Checklist **24**

1. Introduction

Skills turn a capable model into a predictable team member. This guide shows the features a good skill has, and gives a template for each.

The features below were extracted from five QA skills that run a full testing flow: planning from requirements, exploring the running application, writing test cases, reporting bugs and automating tests. Every skill repeats the same building blocks. Reusing them makes new skills faster to write, easier to review and safer to run.

1.1 Agent and skill

	Agent	Skill
File	`.claude/agents/<name>.md`	`.claude/skills/<name>/SKILL.md`
Defines	**What it may use:** tools, model, permission mode	**How it works:** inputs, workflow, rules
Runs	In its own context, returns a report	Inside the session or agent that loads it
Called with	`@agent-<name> <task>`	`/<name>` or picked from its description

Pair every agent with a skill of the same name: the agent restricts the tools, the skill explains the work.

1.2 How to read this guide

- **Chapter 2** shows the folder structure.
- **Chapter 3** presents the 20 skill features. Each one has a short description, the reason it matters and a template.
- **Chapter 4** puts them together into complete templates: skill, agent, reference, output file and lessons.
- **Chapter 5** explains how agents are configured and how to give them access to MCP servers, files and commands.
- **Chapter 6** covers the commands: adding MCP servers and calling an agent.
- **Chapter 7** is a checklist to review a skill before using it.

2. Folder structure

Everything the agents need lives inside the automation folder.

Contents of the automation folder

```
.claude/
├── agents/                          # one file per agent: tools, model, skill
│   ├── qa-planner.md
│   ├── qa-explorer.md
│   ├── qa-test-case-writer.md
│   ├── qa-bug-reporter.md
│   └── qa-automator.md
└── skills/                          # one folder per skill
    ├── qa-planner/
    │   ├── SKILL.md                 # entry point
    │   ├── references/              # loaded only when a step needs them
    │   │   ├── ado-qa-task.md
    │   │   ├── example.md
    │   │   ├── feature-map.md
    │   │   └── test-data-constraints.md
    │   └── scripts/
    │       └── check-feature-map.mjs # deterministic check
    ├── qa-explorer/
    │   ├── SKILL.md
    │   └── references/  (ado-qa-task, environment, fixtures, output-templates)
    ├── qa-test-case-writer/
    │   ├── SKILL.md
    │   └── references/  (ado-test-case-writing, proposed-test-case-format,
    │                     test-case-template)
    ├── qa-bug-reporter/
    │   ├── SKILL.md
    │   └── references/  (ado-bug-reporting, bug-file-format, bug-template, severity)
    └── qa-automator/
        ├── SKILL.md
        └── references/  (ado-automation, framework, test-data-and-cleanup)
.mcp.json                            # MCP servers shared by all agents
.gitignore                           # keeps archive/ and .secrets/ out of git
lessons/                             # one lessons file per agent
└── qa-<agent>-lessons.md
tools/
└── qa-login.mjs                     # signs in without exposing credentials
archive/<WORK_ITEM_ID>/              # agent outputs and hand-offs (git-ignored)
├── QA_PLAN.md
├── TEST_SCENARIOS.md
└── bugs/BUG-NN-<slug>.md
.secrets/                            # local credentials (git-ignored)
fe_automation/                       # Playwright + TypeScript project
├── baseTest/                        # fixtures, global setup, login
├── pageObjects/<portal>/            # one class per page
├── tests/<portal>/*.spec.ts         # specs
├── testData/                        # data factories
├── db/queries/*.sql                 # setup and cleanup queries
└── playwright.config.ts
```

Item	Role
`.claude/agents/`	The **capabilities** of each agent. Short files, frontmatter only.
`.claude/skills/`	The **procedures**. `SKILL.md` plus `references/` and `scripts/`.
`.mcp.json`	The external tools (tracker, database, API contract, browsers) the agents can call.
`lessons/`	What each agent learned from human corrections. Read at the start of every run.
`tools/`	Helpers that do sensitive work (sign-in) so the model never sees secrets.

Item	Role
`archive/`	The shared workspace: one folder per work item, where agents hand work to each other.
`fe_automation/`	The test automation project the automator agent writes into.

> **Start Claude Code from this folder**
> Agents, skills and `.mcp.json` are discovered from the folder Claude Code is started in. Run `claude` inside the automation folder.

3. Skill features

Twenty building blocks that every skill in the set uses. Pick the ones your skill needs and copy the template.

Group	Features
Identity and context	F1 Description contract · F2 Role in the flow · F3 Context block
Knowledge loading	F4 References table · F5 Output templates · F6 Scripts and helpers
Inputs and tools	F7 Command inputs · F8 Tool scoping · F9 Sources of truth
Procedure	F10 Numbered workflow · F11 Progress tracking · F12 Verification · F13 Bounded retries · F14 Final report
Collaboration	F15 Human approval gate · F16 Stable ids and hand-off · F17 Idempotent write-back
Safety and learning	F18 Guardrails · F19 Lessons · F20 Worked example

Identity and context

F1 Description contract

The frontmatter `description` is one dense sentence that says what the skill reads, what it produces, the rule that matters most and where the work is tracked.

Why: Claude chooses skills and agents by this text, and it is the first thing a reviewer reads.

Template

```
---
name: <skill-name>
description: <Reads what> from <source>, <produces what> in <location>,
  <the key rule>, and tracks the work in <task / file> (created if missing,
  set in progress, summary written, then closed).
---
```

Used in: all five skills

F2 Role in the flow

The first paragraph places the skill in the flow: which step comes before, which comes after, and what it deliberately does not do.

Why: Prevents overlap between agents and stops a skill from doing another skill's job.

Template

```
# <Skill Title>

The <Skill Title> is the step after `<previous-skill>`. It <main job>.
It does not <what belongs to another skill>: that is `<other-skill>`'s job.
On the board it only tracks its own run, in the `<Task name> - ...` task.
```

Used in: qa-explorer, qa-bug-reporter, qa-automator

F3 Context block

A few lines (or a small table) with the system under test, the tracker and what is already loaded in context.

Why: The agent starts with the facts it needs and does not waste tokens re-reading files.

Template

```
**System under test:** <app name> - API <url>, UI <url>, database <name>.
**Tracker:** <organisation> / <project>.
`CLAUDE.md` is already in your context - do not Read it again.
Large files: grep the `## ` heading, then Read with offset / limit.
```

Used in: all five skills

Knowledge loading

F4 References table

Details live in `references/`. A table in `SKILL.md` says which file to load at which step.

Why: Keeps `SKILL.md` short (about 120 to 200 lines) and loads knowledge only when it is needed.

Template

```
## References (load only when the step needs it)

| File                                  | When                                 |
|---------------------------------------|--------------------------------------|
| [references/<board>.md](...)          | Steps 0 and N - tracking task        |
| [references/<format>.md](...)         | Step 1 - how to read the input       |
| [references/<template>.md](...)       | Step 4 - output template             |
| [../<other-skill>/references/<x>.md]  | Step 2 - shared with <other-skill>   |
```

Used in: all five skills; feature-map.md is shared between three of them

F5 Output templates

Every file or work item the skill produces has a template in `references/`, with fixed headings and field names.

Why: Outputs look the same on every run, so humans can review them quickly and the next agent can parse them.

Template

```
# references/<output>-template.md

## Title
<Where>: <what goes wrong>          (max 80 characters)

## Steps
1. <action>
2. <action>

## Expected / Actual
- Expected: <result> (source: <requirement>)
- Actual:   <result>

## Evidence
<screenshot path, response excerpt, correlation id>
```

Used in: qa-explorer, qa-bug-reporter, qa-test-case-writer

F6 Scripts and helpers

Checks that must be exact go into `scripts/`. Sensitive actions such as signing in go into a helper in `tools/` that prints only a short result.

Why: A script gives the same answer every time; a helper keeps passwords and tokens out of the model's context.

Template

```
// scripts/check-<thing>.mjs
// Checks that <every entry in X still exists in Y>.
// Usage: node .claude/skills/<skill>/scripts/check-<thing>.mjs
// Exit 0 = in sync, 1 = stale entries (listed), 2 = could not read inputs.

// tools/<login>.mjs
// Reads credentials from .secrets/<file>.env and signs in.
// Prints a one-line result - never the password or the session token.
```

Used in: qa-planner (check-feature-map.mjs), all browser agents (qa-login.mjs)

Inputs and tools

F7 Command inputs

The skill accepts a short list of exact commands: a verb, an id, an optional subset and an optional dry run.

Why: The same grammar for every agent makes them easy to call and to chain.

Template

```
## Inputs
- `<Verb> <ID>`              - every eligible item of that work item.
- `<Verb> <ID> <id> <id>`    - only those items (still only if eligible).
- `<Verb> <ID> dry run`      - read and plan only; no files or board changes.
- A Task id instead of a parent id: the parent is used, and you say so.
- Never ask for credentials: sign in through the helper.
```

Used in: Plan Story · Explore Story · Write test cases · Report bugs · Automate

F8 Tool scoping

For each tool or MCP server: what it may read, what it may write, and an explicit “nothing else”. Add what to do when a tool fails.

Why: Narrows a broad tool to the few actions the skill really needs.

Template

```
## Tools / MCP servers
- `<tracker>` - read: <items>. Write ONLY: <one item type / field>.
  Nothing else: no <tool families>. Auth error -> ask the human to run /mcp.
- `<database>` - read-only SELECT. Never select secret columns.
- `Bash` - only: `<command 1>` and `<command 2>`.
- `Read` / `Grep` - <folders>. `Write` / `Edit` - only <folder> and the lessons file.
- Stop if `<required MCP>` is not available, and say which one.
```

Used in: all five skills

F9 Sources of truth

List the sources the skill may use and which one wins on conflict. For testing skills, forbid looking at the implementation.

Why: Keeps results unbiased: expected behaviour comes from requirements, not from how the code was written.

Template

```
**Black-box.** Expected results come only from: <requirements tool>, <API contract>,
<project docs>. Never open source code, tests, pull requests, commits or git history.
When the acceptance criteria and a document disagree, the acceptance criteria win;
list the difference as a gap. Anything only code could answer -> Open Questions.
```

Used in: qa-planner, qa-explorer

Procedure

F10 Numbered workflow

The procedure is a numbered list starting at 0. Each step does one thing, names the reference it needs and states when to stop.

Why: The agent always knows where it is, and a human can follow or resume the run.

Template

```
## Workflow
0. **Lessons and board.** Read the lessons file. Resolve the work item, find or
   create the tracking task, set it in progress (references/<board>.md).
1. **Collect.** <read inputs>. Nothing found -> say so, stop.
2. **Understand.** <read sources> (references/<x>.md).
3. **Do.** <main action>.
4. **Verify.** <prove the result> (see F12).
5. **Report** to the human; write the summary; close the task (see F11, F14).
```

Used in: all five skills

F11 Progress tracking

Each run is tracked in one dedicated task under the work item: found or created at the start, set in progress, summary written in its description, closed at the end.

Why: The team sees on the board what each agent did, without opening any file.

Template

```
## Tracking task (references/<board>.md)
1. Read the work item with its relations; find the child Task whose title starts
   with `<Task prefix>`. More than one -> use the most recent, list the others.
2. None -> create it as a child of the work item, same area and iteration.
3. Set it to in progress at the start.
4. At the end write the summary into its Description and add a comment:
   `<n> done / <n> failed / <n> blocked`.
5. Close it only if everything succeeded; otherwise leave it open and say why.
```

Used in: qa-planner, qa-explorer, qa-automator

F12 Verification

A step that proves the result before anything is reported: the output was re-read against the rules, tests really ran, no test data was left behind.

Why: “Done” means verified, not just executed.

Template

```
## Verify
- Re-read every output once against the Guardrails before sending it.
- The result really happened: <tests found = expected count>, no skipped tests,
  the assertion fails when the expected value is changed.
- Nothing is left behind: count rows carrying the run tag -> must be 0.
- A result counts only when <precondition, e.g. the app was healthy>.
```

Used in: qa-automator, qa-explorer

F13 Bounded retries

A fixed number of attempts to fix a failure, with a clear rule on what may and may not change.

Why: Stops endless loops and stops the agent from “fixing” a test by hiding a real bug.

Template

```
## When something fails
- At most 3 fix attempts per failure; each attempt is followed by a re-run.
- A fix may change only HOW the step runs (locator, wait, data setup),
  never WHAT is checked (expected value, assertion strength).
- If the failure shows a real defect: stop fixing, leave it failing, report it
  with the evidence.
- After the third attempt: leave it as it is and move on.
```

Used in: qa-automator

F14 Final report

The last step tells the human, in a fixed shape, what was done, what was skipped and what needs their action.

Why: The human can decide the next step in seconds.

Template

```
## Report (to the human)
| Item | Result | Created id | Notes |
|------|--------|------------|-------|
Then list:
- already done (skipped, with id)
- skipped (with status) and invalid (with what to fix)
- blocked items and the human action each one needs
```

Used in: qa-test-case-writer, qa-bug-reporter, qa-explorer

Collaboration

F15 Human approval gate

Outputs that go outside the workspace wait for a human decision written in the file. The next agent acts only on approved items.

Why: A human stays in control of everything that reaches the tracker.

Template

```
## Reviewer Decision
Status: PENDING            <!-- human writes CONFIRMED / ACCEPTED or REJECTED -->
Reviewer note:

# In the next skill's Guardrails:
- Act only on items whose Status is CONFIRMED or ACCEPTED.
- Never act on PENDING, REJECTED or an unclear status; never change a decision.
```

Used in: qa-explorer -> qa-test-case-writer, qa-bug-reporter

F16 Stable ids and hand-off

Every item gets an id that never changes (`FE1`, `BE1`, `BUG-01`), flags use exact spelling, and the output ends with a Handoff section for the next agent.

Why: The next agent matches items by id, not by wording, and knows where to start.

Template

```
FE1. [<role>] <scenario> - <expected result> - <source>
     - `automate: yes|no · regression: yes|no · testcase: yes|no`

### Handoff
- Sign in to: <portals / roles>
- Verify first: <unverified preconditions>
- Run order: <shared-data items together, state-consuming items last>
Ids are never renumbered or reused.
```

Used in: qa-planner -> qa-explorer -> qa-automator

F17 Idempotent write-back

Before creating anything external, check for a duplicate. After creating it, write its id back into the source file.

Why: Running the same command twice never creates the same item twice.

Template

```
1. Skip items that already carry `<id field>:` in the file.
2. Duplicate check: search the tracker for `<unique tag or text>` under the
   work item. Found -> do not create; write its id back; report "already done".
3. Create the item.
4. Write back: `<id field>: <new id>` and `<date field>: <date>` in the file.
   If create succeeded but linking failed, still write back and report it.
```

Used in: qa-test-case-writer, qa-bug-reporter

Safety and learning

F18 Guardrails

A closing list of rules that override everything else in the skill, including lessons.

Why: The few things that must never happen are stated once, clearly, in one place.

Template

```
## Guardrails
- **One write target:** the only thing you may create or change is <...>.
  Everything else is read-only.
- **Credentials:** never read <secrets folder> or <.env>, never print a password
  or token; show <secret ids> only as their last 4 characters.
- **Data:** synthetic, run-tagged data only (`@example.test`); never touch
  records you did not create.
- **Transcribe, do not invent:** every fact comes from the source; gaps are
  listed, never guessed.
- **Hands off:** never edit <application folders>, never reset the environment,
  never commit or push.
```

Used in: all five skills

F19 Lessons

Each skill reads its lessons file at step 0 and appends to it only when a human corrects it.

Why: The agent improves from feedback without the skill itself being rewritten after every run.

Template

```
# lessons/<skill>-lessons.md
Rules learned from user corrections. Read before every run. Newest last.

## YYYY-MM-DD - <short title>
- Lesson: <the rule>
- Why: <what went wrong or what the user said>
- How to apply: <when and how it changes the work>

# In SKILL.md:
A lesson overrides the general guidance, never the Guardrails.
Do not add lessons you inferred yourself.
```

Used in: all five skills

F20 Worked example

The skill ends with one realistic run: the command, what the agent finds, what it writes and the final comment.

Why: One concrete example explains the skill faster than any rule.

Template

```
## Example
User: "<Verb> 123."

The agent finds <n> items: <a> are CONFIRMED, <b> REJECTED. It reads work item 123
(<title>, iteration <x>), finds no duplicates, creates <a> items linked to 123,
writes their ids back into the files, and reports: <a> created, <b> skipped.
```

Used in: all five skills

4. Templates

The features combined into ready-to-copy files.

4.1 SKILL.md

.claude/skills/<skill-name>/SKILL.md

```
---
name: <skill-name>
description: <Reads what>, <produces what>, <key rule>, and tracks the work in
  <task> (created if missing, set in progress, summary written, then closed).
---

# <Skill Title>

The <Skill Title> is the step after `<previous>`. It <main job>. It does not <...>.

**System under test:** <urls>. **Tracker:** <org / project>.
CLAUDE.md is already in your context - do not Read it again.

## References (load only when the step needs it)
| File                    | When        |
|-------------------------|-------------|
| references/<a>.md       | Steps 0, N  |
| references/<b>.md       | Step 3      |

## Inputs
- `<Verb> <ID>` - <default scope>
- `<Verb> <ID> <ids>` - only those items
- `<Verb> <ID> dry run` - read and plan only

## Tools / MCP servers
- `<server>` - read: <...>. Write ONLY: <...>. Nothing else.
- `Bash` - only `<command>`.
- `Write` / `Edit` - only <folder> and the lessons file.

## Workflow
0. Lessons and board: read lessons, resolve the item, task in progress.
1. Collect <inputs>. None -> say so, stop.
2. <step> (references/<b>.md)
3. Verify: re-read against the Guardrails; <proof>.
4. Report to the human; write the summary; close the task.

## Guardrails
- The only thing you may create or change is <...>.
- No credentials; synthetic data only; never edit the application; never commit.

## Lessons
Read `lessons/<skill>-lessons.md` first. Append only when a human corrects you.

## Example
User: "<Verb> 123." -> <what it finds, writes and reports>
```

4.2 Agent file

.claude/agents/<skill-name>.md

```
---
name: <skill-name>
description: <same contract as the skill description>
tools:
  - Read
  - Glob
  - Grep
  - Edit
  - Skill
  - mcp__<server>                # whole server, or mcp__<server>__<tool>
model: opus
effort: high
permissionMode: acceptEdits
skills:
  - <skill-name>
---
```

Field	Meaning
`name`	Used by `@agent-<name>`; same as the skill name
`description`	When Claude should delegate to this agent
`tools`	Allowlist; if omitted, the agent inherits every tool
`model` / `effort`	Model alias and reasoning effort
`permissionMode`	`acceptEdits` runs file edits without a prompt
`skills`	Skills preloaded into the agent's context

4.3 Reference file

.claude/skills/<skill-name>/references/<topic>.md

```
# <Topic> in <tool>

<One paragraph: what this file covers and the only thing the skill may change.>

Tools (`<server>` MCP, always pass `project: "<project>"`):
- read:  `<tool>` (`action`: `get`, `get_batch`)
- write: `<tool>` (`action`: `create`, `update`)

## 1. Resolve the work item
## 2. Find or create <the item>
## 3. Write <the result>
## 4. Close and comment
```

4.4 Output file with an approval gate

archive/<ID>/bugs/BUG-01-<slug>.md

```
---
work_item: <ID>
id: BUG-01
related: []
---
# BUG-01 - <Where>: <what goes wrong>

## Steps to reproduce
1. <action>

## Expected
<result> (source: <requirement>)

## Actual
<result>

## Evidence
<screenshot / response excerpt / correlation id>

## Reviewer Decision
Status: PENDING        <!-- CONFIRMED / ACCEPTED or REJECTED -->
```

5. Agent configuration and access

How an agent is set up, and how to give it access to MCP servers, files and commands.

5.1 Where agents are defined

An agent is a Markdown file with a YAML block at the top. Claude Code loads agents from two places:

Location	Available in	Use it for
`.claude/agents/<name>.md`	This project, for everyone who clones it	Team agents (all QA agents live here)
`~/.claude/agents/<name>.md`	All your projects, only for you	Personal helpers

You can write the file by hand or run `/agents`, which walks you through name, description, tools and model, and lets you edit an existing agent later. When a project agent and a personal agent have the same name, the project one is used.

5.2 Configuration fields

.claude/agents/qa-bug-reporter.md

```
---
name: qa-bug-reporter                # id used by @agent-qa-bug-reporter
description: Reads the bug files ..., reports only the accepted ones as Bug
  work items ..., and records the created Bug id back in each file.
tools:                               # allowlist; omit it to inherit every tool
  - Read
  - Glob
  - Grep
  - Edit
  - Skill
  - mcp__azure-devops                # all tools of one MCP server
model: opus                          # opus | sonnet | haiku | inherit
effort: high                         # reasoning effort
permissionMode: acceptEdits          # how permission prompts behave
skills:                              # skills preloaded into the agent
  - qa-bug-reporter
---
<optional system prompt - empty here, the skill holds the procedure>
```

Field	What it controls	Tip
`name`	The agent's id; `@agent-<name>` uses it	Same name as the skill
`description`	When Claude delegates to the agent	Say what it reads, writes and tracks
`tools`	Which tools the agent may use at all	Give only what the skill needs
`disallowedTools`	Tools removed from an otherwise broad list	Remove risky tools of a server you grant whole
`model`	Model alias, or `inherit` from the session	Use a stronger model for judgement-heavy agents
`effort`	Reasoning effort	`high` for planning and exploration
`permissionMode`	Whether edits / commands ask for approval	See 5.8
`skills`	Skills loaded into the agent's context	One skill per agent keeps it focused
Body	The agent's system prompt	Keep it short or empty when a skill exists

5.3 How the QA agents are configured

All five agents use `model: opus`, `effort: high` and `permissionMode: acceptEdits`, and each preloads the skill with the same name. What differs is the tool list: every agent gets only the tools its job needs.

Agent	File tools	Bash	MCP servers
qa-planner	Read, Write, Edit, Grep	No	azure-devops, swagger, postgres
qa-explorer	Read, Write, Edit, Glob, Grep	Yes (login helper only)	chrome-devtools-mcp, azure-devops, postgres, swagger
qa-test-case-writer	Read, Glob, Grep, Edit	No	azure-devops
qa-bug-reporter	Read, Glob, Grep, Edit	No	azure-devops
qa-automator	Read, Write, Edit, Glob, Grep	Yes (login + test runner)	azure-devops, playwright, chrome-devtools-mcp, postgres, swagger

The two agents that only transcribe approved items to Azure DevOps have no `Write`, no `Bash`, no browser and no database: they can read files, edit one line in them and talk to the tracker, nothing more.

5.4 How access works

A tool call goes through several layers. It succeeds only if every layer lets it through, so you can grant access broadly in one layer and narrow it in another.

#	Layer	Where	Question it answers	Enforced
1	Server definition	`.mcp.json`	Does the MCP server exist?	Yes
2	Server enabled	Approval prompt / `enabledMcpjsonServers`	Is the server switched on in this project?	Yes
3	Agent tools	`tools:` / `disallowedTools:` in the agent file	May this agent use the tool?	Yes
4	Permission rules	`allow` / `ask` / `deny` in `settings.json`	Is this specific call allowed, asked or blocked?	Yes
5	Skill text	Tools and Guardrails in `SKILL.md`	How and when should the agent use it?	No (instruction)

> **Rule of thumb**
> Explain a rule in the skill (layer 5) so the agent understands it, and enforce the same rule in layer 3 or 4 so it holds even when the agent gets it wrong.

5.5 Giving an agent access to an MCP server

Step 1. Define the server

Add it to `.mcp.json` (or with `claude mcp add`, see chapter 6). Its tools are then named `mcp__<server>__<tool>`; `/mcp` lists the exact names.

Step 2. Enable it for the project

.claude/settings.json (or approve the servers when Claude Code asks)

```
{
  "enabledMcpjsonServers": ["azure-devops", "swagger", "postgres"],
  "disabledMcpjsonServers": ["playwright"]
}
```

Step 3. Grant it in the agent file

.claude/agents/<name>.md

```
tools:
  - mcp__swagger                              # every tool of the swagger server
  - mcp__azure-devops__wit_work_item          # only these two tools
  - mcp__azure-devops__wit_work_item_write    #   of the azure-devops server
disallowedTools:
  - mcp__azure-devops__repo_create_branch     # or: grant whole, remove a few
```

Step 4. Allow, ask or block individual tools

.claude/settings.json

```
{
  "permissions": {
    "allow": ["mcp__swagger", "mcp__azure-devops__wit_work_item"],
    "ask":   ["mcp__azure-devops__wit_work_item_write"],
    "deny":  ["mcp__azure-devops__pipelines_run", "mcp__azure-devops__repo_create_branch"]
  }
}
```

Step 5. Describe the allowed use in the skill

SKILL.md, Tools section

```
- `azure-devops` - read: `wit_work_item` (get, get_batch). Write ONLY:
  `wit_work_item_write` (create) for Bug work items. Nothing else.
```

5.6 Giving access to files and folders

File access is controlled by which file tools the agent has, by `Read(...)` / `Edit(...)` rules, and by which folders Claude Code can see.

Need	Agent tools	Rule in settings.json
Read and search files	`Read`, `Glob`, `Grep`	`Read(<glob>)`
Change existing files	`Edit`	`Edit(<glob>)`
Create new files	`Write`	`Edit(<glob>)` (covers writes too)
Never touch a path	(none)	`deny`: `Read(<glob>)` and `Edit(<glob>)`

By default Claude Code only works inside the folder it was started from. The QA agents start in `automation/` but read the application's documentation one level up, so that folder has to be added:

.claude/settings.json

```
{
  "permissions": {
    "additionalDirectories": ["../docs/"],
    "allow": [
      "Read(../docs/**)",
      "Edit(./archive/**)",
      "Edit(./lessons/**)",
      "Edit(./fe_automation/**)"
    ],
    "deny": [
      "Read(./.secrets/**)",
      "Read(./**/.env)",
      "Edit(../backend/**)",
      "Edit(../frontend/**)",
      "Edit(../docs/**)"
    ]
  }
}
```

For a one-off session you can also run `claude --add-dir ../docs` or type `/add-dir ../docs` inside the session. Paths starting with `./` are relative to the folder you started in, `~/` to your home folder, and `//` marks an absolute path.

5.7 Giving access to commands

An agent can run shell commands only if `Bash` is in its tools. Then limit which commands with `Bash(...)` rules, matching the start of the command:

.claude/settings.json

```
{
  "permissions": {
    "allow": [
      "Bash(node tools/qa-login.mjs:*)",
      "Bash(npm --prefix fe_automation test:*)"
    ],
    "deny": [
      "Bash(git push:*)",
      "Bash(docker:*)"
    ]
  }
}
```

5.8 Permission modes

Mode	Behaviour	When to use it
`default`	Asks the first time each tool is used	New or untrusted agents
`acceptEdits`	File edits run without asking; other tools follow the rules	Agents that write files in a known folder (all QA agents)
`plan`	Read-only: it can explore and plan, not change anything	Reviews and dry analysis
`bypassPermissions`	Skips all prompts	Avoid, especially with tracker or database access

A `deny` rule always wins, whatever the mode. That is why secrets and application folders belong in `deny`, not only in the skill.

5.9 Where settings live

File	Scope	In git?
`.claude/settings.json`	Team rules for this project	Yes
`.claude/settings.local.json`	Your personal rules for this project	No
`~/.claude/settings.json`	Your rules for every project	No

- **Precedence between rules:** `deny` > `ask` > `allow`.
- `/permissions` shows the rules in effect and lets you add or remove them.
- `/mcp` shows which servers are connected and their tool names.

5.10 Example: locking down qa-bug-reporter

The bug reporter only has to read bug files, write one line back into them and create Bug work items. This is the full set-up for that job.

```
# .claude/agents/qa-bug-reporter.md
tools:
  - Read
  - Glob
  - Grep
  - Edit
  - Skill
  - mcp__azure-devops__wit_work_item
  - mcp__azure-devops__search_workitem
  - mcp__azure-devops__wit_work_item_write
  - mcp__azure-devops__wit_work_item_link_write
permissionMode: acceptEdits

// .claude/settings.json
{
  "permissions": {
    "allow": ["Edit(./archive/**)", "Edit(./lessons/**)"],
    "ask":   ["mcp__azure-devops__wit_work_item_write"],
    "deny":  ["Read(./.secrets/**)", "Read(./**/.env)"]
  }
}
```

With this, the agent cannot create files, run commands, open a browser or query the database, and Claude Code asks before every Bug it creates. The skill still explains the rest: which bug files count as approved, and that it may only write back the Bug id.

6. Commands

Adding MCP servers and calling an agent.

6.1 Adding an MCP server

Option A: edit `.mcp.json` in the folder Claude Code starts from. It is shared with the team through git.

.mcp.json

```
{
  "mcpServers": {
    "<server-name>": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "<package>@latest", "<arg>"],
      "env": { "<TOKEN_NAME>": "${<ENV_VARIABLE>}" }
    },
    "<remote-server>": {
      "type": "http",
      "url": "https://<host>/mcp"
    }
  }
}
```

- Secrets come from environment variables through `${VAR}`; never write them in the file.
- On Windows, wrap npx: `"command": "cmd"`, `"args": ["/c", "npx", …]`.
- Tool names become `mcp__<server>__<tool>`; these names are used in agent `tools:` lists and permission rules.

Option B: use the CLI.

```
# shared with the team (writes .mcp.json)
claude mcp add --scope project <server-name> -- npx -y <package>@latest

# with an environment variable
claude mcp add --scope project <server-name> -e <TOKEN_NAME>=<value> -- <command> <args>

# remote HTTP server
claude mcp add --transport http <server-name> <https-url>

claude mcp list                 # all servers and status
claude mcp get <server-name>    # details
claude mcp remove <server-name>
/mcp                            # inside a session: status and re-authentication
```

Scope	Stored in	Visible to
`local` (default)	`~/.claude.json`	Only you, this project
`project`	`.mcp.json`	Everyone who clones the repository
`user`	`~/.claude.json`	Only you, all projects

6.2 Calling an agent

```
@agent-<name> <Verb> <ID>

@agent-qa-planner Plan Story <ID>
@agent-qa-explorer Explore Story <ID>
@agent-qa-test-case-writer Write test cases <ID> dry run
@agent-qa-bug-reporter Report bugs <ID> BUG-01 BUG-03
@agent-qa-automator Automate PBI#<ID>
```

- Type `@`, pick the agent (shown as `agent-<name>`), then write the task in the skill's input format (F7).
- The agent runs in its own context and returns a report to your session.
- `/agents` lists and edits agents; `/<skill-name>` runs a skill directly in the current session.

7. Checklist

Review every new skill against this list before running it.

1. The description says what the skill reads, produces and tracks (F1), and matches the agent's description.
2. The first paragraph places the skill in the flow and says what it does not do (F2).
3. `SKILL.md` stays under about 200 lines; details are in `references/`, each tied to a step (F4).
4. Every output has a template (F5); exact checks are scripts (F6).
5. Inputs are exact commands with subset and dry-run variants (F7).
6. Every tool has a read scope, a write scope and a “nothing else” (F8).
7. The workflow is numbered from 0, with stop conditions (F10), tracking (F11) and verification (F12).
8. Retries are bounded (F13) and the run ends with a report (F14).
9. Anything that leaves the workspace passes a human gate (F15) and is written back (F17).
10. Guardrails (F18), a lessons file (F19) and a worked example (F20) are present.
11. The agent's `tools:` list and `settings.json` enforce what the skill text promises (chapter 5).

Canache Gabriel · QA Engineer · Centric Romania